

## Hadoop MapReduce Execution Steps (Single Node Setup)

Required files -

sample\_weather.txt

2023-01-01,30,20,10

2023-01-02,32,22,12

2023-01-03,28,18,8

2023-01-04,35,25,15

2023-01-05,31,21,11

WeatherAnalysis.java

```
import java.io.IOException;
```

```
import org.apache.hadoop.conf.Configuration;
```

```
import org.apache.hadoop.fs.Path;
```

```
import org.apache.hadoop.io.DoubleWritable;
```

```
import org.apache.hadoop.io.IntWritable;
```

```
import org.apache.hadoop.io.Text;
```

```
import org.apache.hadoop.mapreduce.Job;
```

```
import org.apache.hadoop.mapreduce.Mapper;
```

```
import org.apache.hadoop.mapreduce.Reducer;
```

```
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
```

```
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```
public class WeatherAnalysis {
```

```
public static class WeatherMapper
    extends Mapper<Object, Text, Text, DoubleWritable>{

    private Text type = new Text();
    private DoubleWritable value = new DoubleWritable();

    public void map(Object key, Text line, Context context)
        throws IOException, InterruptedException {

        String[] fields = line.toString().split(",");

        double temp = Double.parseDouble(fields[1]);
        double dew = Double.parseDouble(fields[2]);
        double wind = Double.parseDouble(fields[3]);

        type.set("Temperature");
        value.set(temp);
        context.write(type, value);

        type.set("DewPoint");
        value.set(dew);
        context.write(type, value);

        type.set("WindSpeed");
        value.set(wind);
```

```
        context.write(type, value);
    }
}
```

```
public static class WeatherReducer
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable>{

    public void reduce(Text key, Iterable<DoubleWritable> values,
        Context context)
        throws IOException, InterruptedException {

        double sum = 0;
        int count = 0;

        for(DoubleWritable val : values){
            sum += val.get();
            count++;
        }

        double avg = sum / count;

        context.write(key, new DoubleWritable(avg));
    }
}
```

```
public static void main(String[] args) throws Exception {
```

```
Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "Weather Analysis");

job.setJarByClass(WeatherAnalysis.class);

job.setMapperClass(WeatherMapper.class);
job.setReducerClass(WeatherReducer.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(DoubleWritable.class);

FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

## 1. Start Hadoop DFS Services

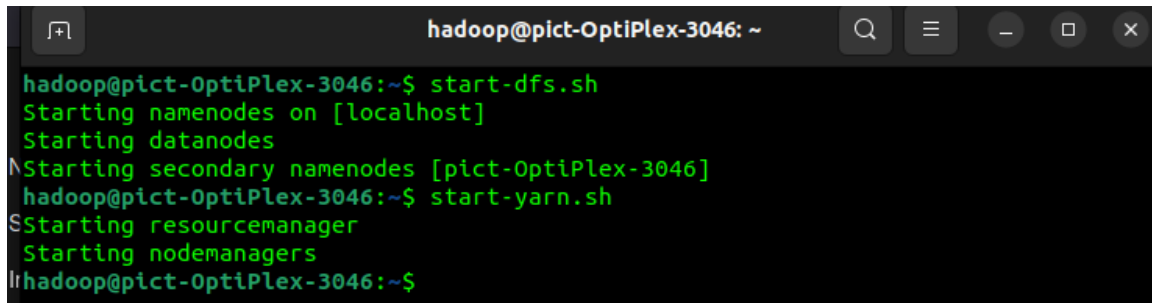
Run the following command to start HDFS services (NameNode, DataNode, SecondaryNameNode):

```
start-dfs.sh
```

## 2. Start YARN Services

Run the following command to start YARN services (ResourceManager and NodeManager):

```
start-yarn.sh
```

A terminal window titled 'hadoop@pict-OptiPlex-3046: ~' with search, menu, and window control icons. It shows the execution of 'start-dfs.sh' and 'start-yarn.sh'. The first command starts namenodes on localhost, datanodes, and secondary namenodes. The second command starts the ResourceManager and NodeManagers.

```
hadoop@pict-OptiPlex-3046:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [pict-OptiPlex-3046]
hadoop@pict-OptiPlex-3046:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@pict-OptiPlex-3046:~$
```

## 3. Check Running Hadoop Processes

Check if all Hadoop services are running using:

```
jps
```

Expected processes:

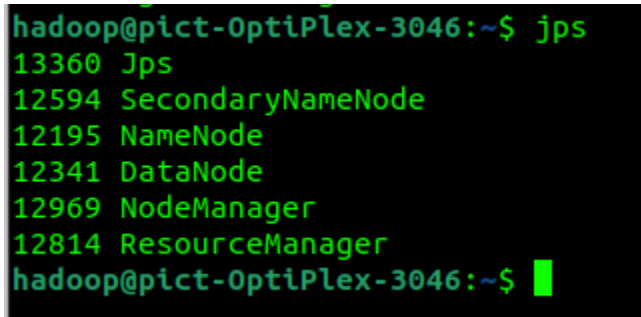
NameNode

DataNode

SecondaryNameNode

ResourceManager

NodeManager

A terminal window showing the output of the 'jps' command. It lists six running processes with their respective PIDs: Jps (13360), SecondaryNameNode (12594), NameNode (12195), DataNode (12341), NodeManager (12969), and ResourceManager (12814).

```
hadoop@pict-OptiPlex-3046:~$ jps
13360 Jps
12594 SecondaryNameNode
12195 NameNode
12341 DataNode
12969 NodeManager
12814 ResourceManager
hadoop@pict-OptiPlex-3046:~$
```

#### 4. If NameNode is Missing (Fix)

If NameNode is not visible in the jps output, format the NameNode using:

```
hdfs namenode -format
```

Then start Hadoop again:

```
start-dfs.sh
```

```
start-yarn.sh
```

#### 5. Compile the Java MapReduce Program

```
javac -classpath `hadoop classpath` -d . WeatherAnalysis.java
```

#### 6. Create the JAR File

```
hadoop@pict-OptiPlex-3046:~$ javac -classpath `hadoop classpath` -d . WeatherAnalysis.java
hadoop@pict-OptiPlex-3046:~$ jar -cvf weather.jar *.class
added manifest
adding: WeatherAnalysis$WeatherMapper.class(in = 1935) (out= 811)(deflated 58%)
adding: WeatherAnalysis$WeatherReducer.class(in = 1722) (out= 732)(deflated 57%)
adding: WeatherAnalysis.class(in = 1503) (out= 808)(deflated 46%)
adding: WordCount$IntSumReducer.class(in = 1755) (out= 750)(deflated 57%)
adding: WordCount$TokenizerMapper.class(in = 1752) (out= 761)(deflated 56%)
adding: WordCount.class(in = 1472) (out= 805)(deflated 45%)
hadoop@pict-OptiPlex-3046:~$
```

```
jar -cvf weather.jar *.class
```

#### 7. Prepare HDFS Input Directory

```
hdfs dfs -mkdir /input
```

```
hdfs dfs -put sample_weather.txt /input
```

#### 8. Run the MapReduce Job

```
hadoop jar weather.jar WeatherAnalysis /input /output
```

#### 9. Display Output

```
hdfs dfs -cat /output/part-r-00000
```

```
hadoop@pict-OptiPlex-3046:~$ hdfs dfs -cat /output/part-r-00000
DewPoint      21.2
Temperature    31.2
WindSpeed     11.2
hadoop@pict-OptiPlex-3046:~$
```